BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

ref	https://arxiv.org/pdf/1810.04805
type	예습과제
week	week5

BERT

: 트랜스포머 기반 양방향 인코더를 사용하는 언어 처리 모델

- 양방향 인코더 : 입력 시퀀스를 양쪽 방향에서 처리 >> 이전과 이후 단어 모두 참조하여 더 정확하게 단어의 의미와 문맥 파악 가능
- 이미 대규모 데이터를 사용해 사전 학습되어있음 & 전이 학습에 주로 활용 OR 재사용하여 적은 양의 data로도 높은 정확도 달성 ㄱㄴ

[트랜스포머의 인코더를 기반으로 하는 BERT 모델]

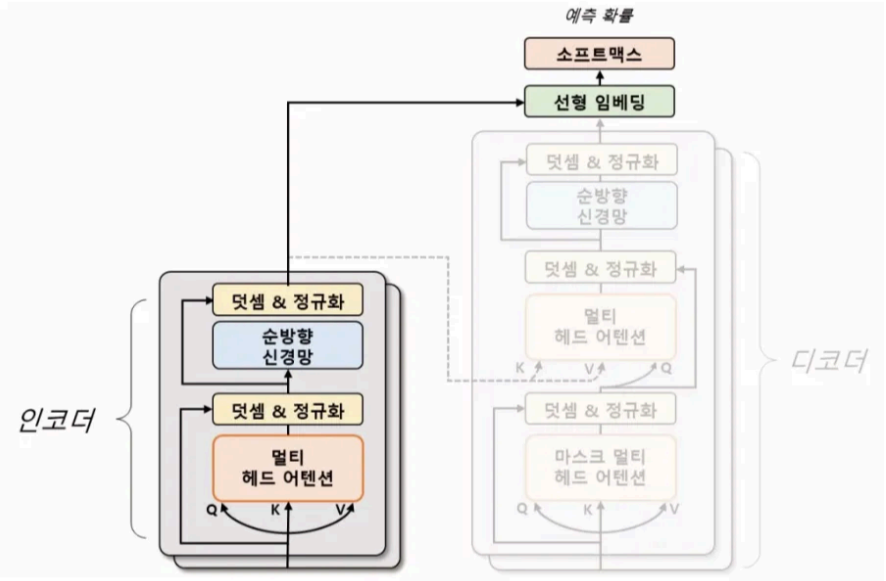


그림 7.13 트랜스포머와 BERT 모델 비교

- 인코더 모듈만 사용 >> 입력 문장의 단어들을 임베딩 & 의미를 벡터화 >> 순차적으로 처리하여 문장 전체의 의미를 추출
- 사전 학습으로 마스킹된 언어 모델링 MLM과 다음 문장 예측 방법 NSP 사용

사전 학습 방법

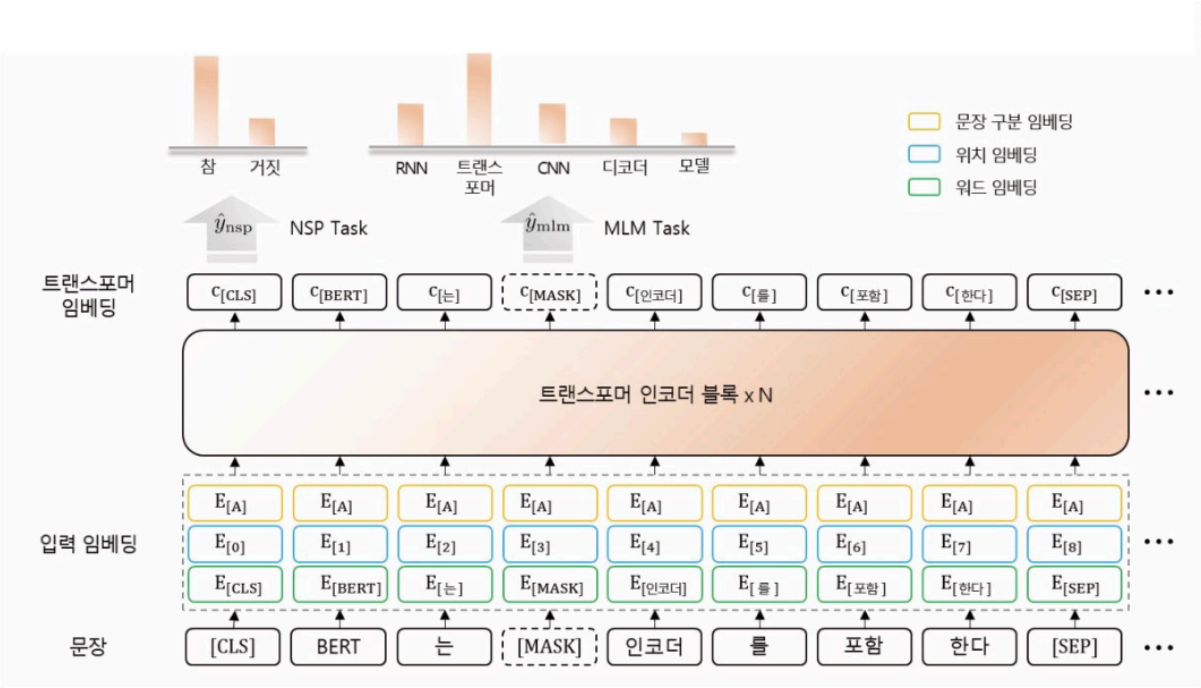
1. MLM과 NSP

- MLM : 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식, 양방향 문맥 정보 참고
    - ex) I'm learning Pytorch(learning을 마스킹) → I'm [MASK] Pytorch
- NSP : 두 개의 문장을 받았을 때 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단
    - BERT는 두 문장의 연속 관계 예측 및 관계 학습&의미적 유사성 파악

2. 토큰 : 특수 토큰을 추가해 모델의 학습과 추론에 정보를 제공

- [CLS] : 입력 문장의 시작 부분에 추가되는 토큰 → 입력 문장의 유형 미리 파악 가능
- [SEP] : 두 개의 문장을 구분할 때 사용하는 토큰
- [MASK] : 임의로 선택된 단어를 나타내는 토큰 → 단어 예측에 활용

⇒ [CLS] 문장1 [SEP] 문장2 [MASK] [SEP] 이런 형태



- BERT는 사전 학습 후 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용 가능

모델 실습

1. Korpora 라이브러리로 데이터 불러오기

```
import numpy as np
import pandas as pd
from Korpora import Korpora


corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=4
2)

train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(d
f)), int(0.8 * len(df))]
)

print(train.head(5).to_markdown())
```

```
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

2. BertTokenizer 클래스로 데이터 전처리

```
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampl
er


def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.lo
ng).to(device)
    return TensorDataset(input_ids, attention_mask, labels)


def get_datalodader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler,
batch_size=batch_size)
    return dataloader


epochs = 5
batch_size = 32
device = "mps" if torch.backends.mps.is_available() else "c
pu"
```

```
print("디바이스 설정:", device)

tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-c
ased", #사전학습된 도멜에 다중 언어 지원 > 대소문자 유지하는 사전
학습된 BERT 모델
    do_lower_case=False #대소문자 구분 X
)

train_dataset = make_dataset(train, tokenizer, device) #텐
서 데이터세트 반환
train_dataloader = get_datalodader(train_dataset, RandomSam
pler, batch_size) #샘플러 클래스로 데이터 샘플링
                                                    #무작위로 샘
플링 > 학습에 적용

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_datalodader(valid_dataset, Sequentia
lSampler, batch_size) #SequentialSampler 데이터를 고정된 순서
대로 반환 > 검증 및 평가 배치

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_datalodader(test_dataset, SequentialS
ampler, batch_size)

print(train_dataset[0])
```

- do_lower_case=False : 대소문자 구분 X
- RandomSampler : 무작위로 샘플링 > 학습에 적용 & 샘플러 클래스로 데이터 샘플링
- SequentialSampler 데이터를 고정된 순서대로 반환 > 검증 및 평가 배치

3. BertForSequenceClassification로 모델 선언

```
from torch import optim
from transformers import BertForSequenceClassification

# 패딩기법사용 > 모델 변경 X
model = BertForSequenceClassification.from_pretrained(
```

```
    pretrained_model_name_or_path="bert-base-multilingual-c
ased",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e
-8) #활성화 함수, 가중치 감쇠를 추가함
```

- 토큰 임베딩 + 위치 임베딩 + 문장 구분 임베딩으로 구성

```
bert
└ embeddings
│  └ word_embeddings
│  └ position_embeddings
│  └ token_type_embeddings
│  └ LayerNorm
│  └ dropout
└ encoder
│  └ layer
│  │  └ 0
│  │  └ 1
│  │  └ 2
│  │  └ 3
│  │  └ 4
│  │  └ 5
│  │  └ 6
│  │  └ 7
│  │  └ 8
│  │  └ 9
│  │  └ 10
│  │  └ 11
└ pooler
│  └ dense
│  └ activation
dropout
classifier
```

4. 모델 학습 및 평가

▼ Teriminology

1. MLM : input token에 무작위로 마스킹, context에 근거하여 마스킹된 단어 예측
2. Feature-based Approaches: 사전 학습된 표현(Representation)을 고정된 특징(Feature)으로 추출하여 기존의 작업별(Task-specific) 모델의 입력값으로 활용하는 방식 ex)ELMo
3. Fine-tuning Approaches: 사전 학습된 모델의 파라미터를 특정 작업(Downstream Task) 학습 시 전체적으로 업데이트하여 최적화하는 방식 ex) OpenAI의 ChatGPT

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

BERT : NLP에 있어 사전 학습된 deep bidirectional 트랜스포머 언어 모델

해당 task에서 기존 연구 한계점

- fine-tuning에 있어 pre-trained된 representations의 효과를 제한 → pre-training에서의 단방향 구조
- Sub-optimal for Sentence-level Tasks과 같은 문장 전체 구조 파악 테스크에서 성능 저하
- Token-level Tasks에서 특히 harmful : 앞뒤 문맥 파악이 필수지만 구조상 불가능

논문의 contributions

BERT

- 기존의 양방향성 제약을 MLM을 통해 해소→deep 양방향 representations에 활용
- 다양한 벤치마크 평가를 통해 NLP에 있어 BERT의 우수성을 정량적으로 증명

2. Related Work

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

기존 연구 및 차별성

- Unsupervised Feature-based → Deep vs. Shallow Bidirectional (like ELMo)
  - 초기 연구는 단어 수준의 Word Embedding(Word2Vec, GloVe)에 집중했으나, 이후 문장 및 단락 단위로 확장
  - ELMo : L-to-R과 R-to-L LSTM을 각각 학습시켜 단순히 concat하여 context-sensitive features를 제공
- Unsupervised Fine-tuning → Bidirectional vs. Unidirectional (like GPT)
  - 레이블이 없는 텍스트로 인코더를 학습시킨 후, 특정 작업에 맞춰 전체 파라미터를 조정하는 방식
  - OpenAI GPT가 대표적 but L-to-R LM 구조에 의존
  - BERT는 MLM을 사전 학습 목표로 적용, 양방향 문맥 활용&토큰의 자기 참조 해결

- Transfer Learning from Supervised Data
  - NLI나 기계 번역과 같은 대규모 레이블 데이터셋에서 학습된 모델을 전이하는 방식,CV 분야에서 ImageNet 사전 학습 모델을 사용하는 것과 유사한 전략
- Architecture Simplicity
  - 특징 기반 방식Ex) ELMo과 달리, BERT는 복잡한 작업별 아키텍처를 추가할 필요 X, 매우 적은 수의 파라미터만 처음부터 학습→ 효율, 범용성

3. 제안 방법론

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

MLM(Masked LM) : Token의 random한 확률에 따라 mask→masked된 것만 예측

- mismatch in fine-tuning의 개선 : 15%의 mask 처리시 → 80%는 [mask] + 10%는 random token + 10%는 안 바꿈

NSP(Next Sentence Prediction) : QA나 NLI 같은 두 문장간의 관계를 이해해야하는 task

- in pre-train : 단일 corpus 생성 → 50%는 다음 문장, 50%는 다른 랜덤 문장 ⇒ [CLS]가 NSP에 사용

모든 파라미터 전달 <> 기존 : 문장 임베딩만 전달

pre-training corpus : document-level corpus가 sentence level shuffle보다 성능 좋음

Contribution

- SOTA 달성: 11개의 NLP 태스크(GLUE, SQuAD v1.1/v2.0 등)에서 당시 기존 기록을 능가&SOTA
- BERT-Large: 모델의 크기가 커질수록(Layer 수, Hidden size 증가) 데이터가 적은 태스크에서도 성능 향상이 뚜렷하게 나타남을 입증

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset & Baseline & 결과

Dataset	Baseline	결과
GLUE	OpenAI GPT, BiLSTM+ELMo 등	BERT-L는 평균 80.5(GPT는 72.8), 4.6%의 정확도 향상
SQuAD v1.1	BiDAF, ELMo 등	단일 모델만으로도 outperform, TriviaQA로 사전학습 → F1에서 최고 점수
SQuAD v2.0	SAN, U-Net, SLQA+ 등 기존 SOTA	BERT만 간단히 사용해도 F1 점수가 5.1점 향상
SWAG	ESIM+ELMo, OpenAI GPT, 인간	BERT-L가 86.3%를 기록, Baseline인 ESIM+ELMo 대비 +27.1%, GPT 대비 +8.3% 향상, 인간(85.0%)에 근접

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

의의

- 단방향 구조의 한계 타파, pre-train의 새로운 구조 제시
- 강력한 사전 학습 모델 + 간단한 output layer → 범용성 및 효율성
- 대규모 unlabeled dataset들로 사전 학습 → 소규모 labeled dataset에서 고성능

한계

- [Mask] 토큰의 불일치는 그래서 해결된거?
- 연산 자원량;;